

Metody Programowania

Zadania do treningu

Data aktualizacji: 2026-06-17

Tablica zawartości

I	Przedmowa	3
I.1	Format zawartości	3
II	Wiązanie zmiennych	4
III	Typowanie wyrażeń	5
IV	Rekurencja ogonowa	6
V	Identyfikacja funkcji	7
VI	Indukcja strukturalna	8
VI.1	Formułowanie zasady indukcji	8

I. *Przedmowa*

Materiału do przedmiotu *Metod Programowania* nie da się nauczyć w tydzień. Nawet dwa tygodnie nie wystarczą. Najambitniejsi być może zdołają to uczynić w 2 tygodnie i 42 sekundy. Przedmiot ten wymaga systematycznej pracy, programowania w OCamlu na co dzień, realizowania zadań z list i pracowni, zadawania pytań. Zainteresowanie tematem języków programowania może usprawnić systematyczną pracę, pozwolić głębiej zrozumieć problemy zadawane na zajęciach.

Innymi słowy – jeżeli za $<$ tydzień masz egzamin i nie wiesz o czym jest ten przedmiot, to lepiej zmienić studia lub przygotowywać się do powtarzania MP. **Natomiast**, jeżeli skrupulatnie się uczyłeś, chodziłeś na wykłady i na nich uważałeś, próbowałeś podchodzić do większości zadań z list – nie masz raczej czego się bać. Ten zestaw pytań to będzie dla Ciebie jedynie powtórka z materiału całego wykładu, usystematyzowanie wiedzy, ćwiczenia mentalne. Na spokojnie można to przejrzeć dzień przed egzaminem, spróbować porozwijać parę zadań samodzielnie, a resztę dnia odpoczywać.

Jeśli jednak masz problemy z $>$ połową zadań, to na naukę zapewne za późno, ale warto próbować. Powodzenia!

I.1 *Format zawartości*

Każde zadanie jest wydrukowane na osobnej kartce, dając miejsce na samodzielne rozwiązanie/nie spojlerując odpowiedzi. Na stronach kolejnych są wszelkie wskazówki do zadania, które mogą lekko pomóc w jego rozwiązaniu. Wreszcie – rozwiązanie z wyjaśnieniem.

Ponownie, na naukę widząc ten materiał parę dni przed sesją, pewnie już za późno. Tutaj zakładam że większość rzeczy jest znane, tłumacząc tylko rzeczy mniej widoczne dla niewprawionego oka.

II. Wiązanie zmiennych

W poniższych wyrażeniach podkreśl wolne wystąpienia zmiennych. Dla każdego związanego wystąpienia zmiennej, narysuj strzałkę od tego wystąpienia do wystąpienia wiążącego je.

```
let f x =  
  
    let x = x  
  
    and y = x * y in  
  
f x y z
```

```
fun f x y ->  
  
    let z = x + y in  
  
    let x = y + x in  
  
fun y -> g x y z
```

```
let rec f x =  
  
    let rec g z y = j y (f z)  
  
    and j y z = g z (f y)  
  
in  
  
if g y > f x then  
  
    g x  
  
else  
  
    g y
```

```
let fun x = x * x in  
  
(fun x -> x * 5) y  
  
|> (fun x -> y)  
  
|> x
```

```
let zagadka b c f =  
  
    match b with  
  
    | [] -> []  
  
    | a :: b ->  
  
        zagadka b (f a c) f
```

```
let z = function  
  
    | [] -> 0  
  
    | _ :: d -> 1 + (z d)  
  
and f = z  
  
and g xs ys =  
  
    (f xs) * (f ys)
```

III. Typowanie wyrażeń

Dla poniższych wyrażeń w języku OCaml podaj ich (najogólniejszy) typ, lub napisz BRAK TYPU, gdy wyrażenie nie posiada typu (nie typuje się)

Lp	Wyrażenie	Typ
1.	<code>fun x -> x</code>	
2.	<code>fun x -> x *. 2</code>	
3.	<code>fun x y -> !x > (y+2,10)</code>	
4.	<code>let rec add a b = if b = 0 then a else add (a + 1) (b-1)</code>	
5.	<code>fun f x -> f (f x)</code>	
6.	<code>fun f -> (fun x -> f x x)(fun y -> f y y)</code>	
7.	<code>fun f x -> f (f x) > x</code>	
8.	<code>fun a b c f d g e -> f b d c (g a)</code>	
9.	<code>List.fold_right</code>	
10.	<code>List.map</code>	
11.	<code>List.iter2</code>	
12.	<code>fun f x y -> f</code>	

IV. Rekurencja ogonowa

Określ czy definicje poniższych funkcji rekurencyjnych są ogonowe. Jeżeli tak, wpisz w komórce obok TAK, jeżeli nie – przepisz je na ogonowe, lub uzasadnij, dlaczego to niemożliwe.

Lp	Definicja funkcji	Odpowiedź
1.	<pre>let rec f a = if a < 0 then -1 else if a = 0 then a else a + f (a-1)</pre>	
2.	<pre>let rec comp f fil a = if fil a then a else comp f fil (f a)</pre>	
3.	<pre>let rec f g xs = match xs with [] -> g 0 x :: xs -> (g x) :: (f g xs)</pre>	
4.	<pre>let rec app xs ys = match xs with [] -> ys x :: xs -> x :: (app xs ys)</pre>	
5.	<pre>let rec comp f fil a = if fil a then a else 1 + comp f fil (f a)</pre>	
6.	<pre>let rec f g xs acc = match xs with [] -> acc x :: xs -> f g xs (g acc x)</pre>	

V. Identyfikacja funkcji

W każdym rzędzie podana jest funkcja w postaci typu, definicji i jej działania. Wypełnij brakujące informacje.

Pierwszy wpis (dla `square`) jest podany jako przykład.

Lp	Typ funkcji	Definicja funkcji	Opis funkcji
1.	<code>int -> int</code>	<code>fun a -> a * a</code>	Podnosi argument <i>a</i> do kwadratu.
2.	<code>int -> 'a -> 'a list</code>		Wstawia <i>n</i> elementów <i>a</i> na początek listy <i>xs</i> .
3.		<code>fun a b c -> if a > 0 then b a else c a</code>	
4.	<code>(ident * expr) list -> ident -> expr -> (ident * expr) list</code>		Aktualizuje (jeżeli para z pierwszą współrzędną równą <i>ident</i> jest już w liście, to ją najpierw usuwa) listę par wprowadzając nową parę (<i>ident</i> , <i>expr</i>).
5.			Zamienia listę elementów typu τ <i>opt</i> na listę elementów typu τ , pozbywając się pustych elementów.

VI. *Indukcja strukturalna*

VI.1 *Formułowanie zasady indukcji*

Rozważmy następujący typ danych reprezentujący wyrażenia złożone z zer, operacji następnika i dodawań.

```
type expr =  
  | Zero  
  | Succ of expr  
  | Add of expr * expr
```

Sformułuj zasadę indukcji dla tego typu danych.